

INHERITED FROM Helix Constitution

This module is a submodule of a project that includes the Helix Constitution submodule. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references apply unconditionally. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution#CLAUDE.md> - LLMProvider Module

Definition of Done

This module inherits the parent project's universal Definition of Done – see the root `CLAUDE.md` and `docs/development/definition-of-done.md`. In one line: **no task is done without pasted output from a real run of the real system in the same session as the change**. Coverage and green suites are not evidence.

Acceptance demo for this module

```
# Circuit breaker + health monitor + retry policy for provider fault toler
cd LLMProvider && GOMAXPROCS=2 nice -n 19 go test -count=1 -race -v \
    -run 'TestDefaultCircuitBreakerConfig|TestHealthMonitor_|TestDefaultRetr
```

Expect: PASS; breaker opens after 3 consecutive failures, recovers after cooldown.
`LLMProvider/README.md` shows the full `LLMProvider` interface.

Overview

`digital.vasic.llmprovider` is a generic, reusable Go module providing LLM provider abstractions and utilities. It defines the core `LLMProvider` interface and common patterns for building LLM provider implementations, including circuit breakers, health monitoring, retry logic, and lazy loading. The module is designed for AI/LLM applications that need to integrate multiple LLM providers with fault tolerance and observability.

Module: `digital.vasic.llmprovider` (Go 1.25+) **Dependencies:**
`digital.vasic.models`, `github.com/sirupsen/logrus` **Test Dependencies:**
`github.com/stretchr/testify`

Build & Test

```
go build ./...
go test ./... -count=1 -race
go test ./... -short                # Unit tests only
```

Code Style

- Standard Go conventions, `gofmt` formatting
- Imports grouped: `stdlib`, third-party, internal (blank line separated)
- Line length $\approx$ 100 characters

- Naming: camelCase private, PascalCase exported, acronyms all-caps
- Errors: always check, wrap with `fmt.Errorf("...: %w", err)`
- Tests: table-driven, `testify`, naming `Test<Struct>_<Method>_<Scenario>`

Package Structure

Package	Purpose
<code>llmprovider</code> (root)	Core types: <code>LLMProvider</code> interface, circuit breaker, health monitor, retry config, lazy provider, and associated utilities

Key Interfaces

- `LLMProvider`: Interface for LLM provider implementations with `Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig`
- `CircuitBreaker`: Wraps an `LLMProvider` with fault tolerance (closed/open/half-open states)
- `HealthMonitor`: Tracks provider health with configurable thresholds and intervals
- `RetryConfig`: Configurable retry logic with exponential backoff and jitter
- `LazyProvider`: Lazy initialization of providers with optional event publishing

Core Components

LLMProvider Interface

The foundational interface that all LLM provider implementations must satisfy:

```
type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest) (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest) (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
    ValidateConfig(config map[string]interface{}) (bool, []string)
}
```

Circuit Breaker

Prevents cascading failures when providers are unhealthy: - **Closed**: Normal operation, requests pass through - **Open**: Provider is failing, requests are short-circuited - **Half-Open**: Testing if provider has recovered

Health Monitor

Tracks provider health with: - Configurable check intervals and timeouts - Consecutive failure/success thresholds - Health status transitions (healthy, degraded, unhealthy, unknown) - Listener support for health status changes

Retry Logic

Configurable retry with: - Exponential backoff with configurable multiplier - Jitter to prevent thundering herd - HTTP status code detection (429, 500, 502, 503, 504) - Context cancellation support

Lazy Provider

Lazy initialization pattern: - Deferred provider initialization until first use - Configurable timeout and retry attempts - Optional event bus integration for provider lifecycle events

Dependencies

- **digital.vasic.models**: For LLMRequest, LLMResponse, ProviderCapabilities types
- **github.com/sirupsen/logrus**: For structured logging in circuit breaker
- **Standard library**: context, sync, time, net/http, etc.

Thread Safety

- CircuitBreaker, HealthMonitor, and CircuitBreakerManager are thread-safe using sync.RWMutex
- RetryConfig is immutable after creation
- LazyProvider is thread-safe for concurrent initialization
- All exported methods are safe for concurrent use unless otherwise documented

Example Usage

```
import (
    "context"
    "digital.vasic.llmprovider"
    "digital.vasic.llmprovider/pkg/models"
)

func main() {
    provider := // create your provider implementation
    cb := llmprovider.NewDefaultCircuitBreaker("my-provider", provider)

    req := &models.LLMRequest{
        Prompt: "Hello, world!",
        MaxTokens: 100,
    }

    resp, err := cb.Complete(context.Background(), req)
    if err != nil {
        log.Fatal(err)
    }

    fmt.Println(resp.Text)
}
```

Integration with the parent project

This module is extracted from the parent project's `internal/llm` package. In the parent project, provider implementations (Claude, DeepSeek, Gemini, etc.) implement the `LLMProvider` interface and use these utilities for fault tolerance and observability.

Integration Seams

Direction	Sibling modules
Upstream (this module imports)	Models
Downstream (these import this module)	DebateOrchestrator, HelixLLM

Siblings means other project-owned modules at the parent project repo root. The root parent-project app and external systems are not listed here – the list above is intentionally scoped to module-to-module seams, because drift *between* sibling modules is where the “tests pass, product broken” class of bug most often lives. See root `CLAUDE.md` for the rules that keep these seams contract-tested.